



VIT[®]
BHOPAL



VITYARTHI STUDENT PROJECT REPORT

Study Buddy

EXAM READINESS ANALYZER

A menu-driven Python command-line application that tracks attendance, internal marks and weekly study hours to report subject-wise exam readiness.

● PYTHON 3

● 4 MENU OPTIONS

● 3 READINESS RULES

SUBMITTED BY
H. S. Preetham

COLLEGE
VIT Bhopal University

COURSE
Python Essentials (VITyarthi)

GUIDED BY
Ms. Vaishnavi S

SUBMITTED
September 2026



Acknowledgement

I would like to express my sincere gratitude to **VIT Bhopal University** for providing the platform, resources and structured learning environment that made this project possible. The exposure to practical, hands-on programming through the **VITyarthi** ecosystem has been instrumental in helping me apply classroom concepts to a real, working application.

I am especially thankful to my mentor, **Ms. Vaishnavi S**, for the continuous guidance, constructive feedback and encouragement extended throughout the duration of the **Python Essentials** course. Her support in clarifying core Python concepts — from file handling to exception handling — helped shape the design and logic of this project.

I also extend my thanks to the VITyarthi team for curating a course structure that balances theory with project-based assessment, giving students like me the opportunity to translate learning into a tangible, portfolio-ready application.

Finally, I am grateful to my family, friends and peers for their patience and encouragement during the planning, coding, debugging and documentation of this project. This report and the accompanying source code are a reflection of that combined support.

— *H. S. Preetham*

CERTIFICATE

Bonafide Project Work Declaration

This is to certify that the project report entitled
“STUDY BUDDY – EXAM READINESS ANALYZER”

is a bonafide record of work carried out by H. S. Preetham, submitted in partial fulfilment of the requirements of the course “Python Essentials”, undertaken on the VITyarthi learning platform, under the guidance of Ms. Vaishnavi S, VIT Bhopal University.

The matter embodied in this report has not been submitted elsewhere for the award of any other credit and represents the original, independent work of the author, developed and tested using Python 3.



Ms. Vaishnavi S
Project Guide / Mentor

Course Coordinator
VITyarthi, VIT Bhopal University

Place: Bhopal, Madhya Pradesh

Date: September 2026

Table of Contents

Acknowledgement	2
Certificate (Bonafide Declaration)	3
1. Introduction & Project Overview	5
2. Objectives & Key Features	6
3. Readiness Logic & Evaluation Rules	7
4. Technologies & Python Concepts Used	8
5. System Design — Program Flow	9
6. Project Structure & Data Storage Format	10
7. Sample Execution & Output	11
8. Error Handling & Testing Notes	12
9. Limitations of the Current Version	13
10. Future Scope	14
11. Conclusion	15
Annexure — VITyarthi Course Completion Certificate	16

1 Introduction & Project Overview

Study Buddy – Exam Readiness Analyzer is a command-line Python application built to help students keep track of their academic readiness ahead of examinations. Instead of mentally juggling attendance percentages, internal marks and study plans across several subjects, the program brings all three data points together and turns them into one simple, easy-to-read status.

For every subject, the user enters the current **attendance percentage**, the **internal marks** (out of 100) and the number of **study hours planned for the week**. The program then checks these values against three straightforward rules and reports how many of the rules are satisfied, along with a friendly readiness message such as *“Ready for exam!”* or *“Not good, need more efforts”*.

All subject data is stored in a plain text file, `subjects.txt`, so that the information a student enters in one session is still available the next time the program is run — without needing a database or any external service.

This project was built as part of the **Python Essentials** course on **VITyarthi**, and its primary purpose is academic: to demonstrate, through a single practical application, how fundamental Python building blocks — variables, lists, functions, loops, conditionals, string handling and file I/O — combine to solve a problem a student can immediately relate to.

Why this problem?

Attendance shortfalls, low internal marks and poor time-management are among the most common, and most preventable, reasons students struggle before an exam. A lightweight tool that surfaces this information clearly — subject by subject — encourages timely corrective action rather than a last-minute scramble.

Aspect	Detail
Interface	Command-line (terminal), menu-driven
Language	Python 3 — uses only the standard library
Data storage	Plain text file — <code>subjects.txt</code>
Core idea	Score each subject against 3 readiness rules (0–3 points)

2 Objectives & Key Features

Objectives

- Store subject-wise academic information (attendance, marks, study hours) in one place.
- Track attendance, internal marks and planned weekly study hours per subject.
- Evaluate a student's exam readiness using clear, simple conditions.
- Present an easy-to-understand readiness status for every subject.
- Persist data between different runs of the program using file handling.
- Demonstrate fundamental Python programming concepts through a practical, working application.

Key Features

 Add Subject Enter a subject's attendance, marks and planned study hours. If the subject already exists, choose whether to update it.	 View Subjects Display every saved subject in a clean tabular format — name, attendance, marks and hours.	 Check Readiness Run the three readiness rules against each subject and print a status with the number of conditions met.	 Save & Exit Persist all subject records to subjects.txt and close the program safely.
--	--	--	---

Together, these four menu options form a complete, self-contained loop: a student can add and update subjects across multiple sittings, review what has been entered so far, get an instant readiness verdict, and save everything safely before quitting.

3 Readiness Logic & Evaluation Rules

The heart of the application is a simple point-based scoring system. Every subject is checked against three independent conditions; each satisfied condition contributes exactly one point, for a maximum possible score of three.

#	Condition	Passing Requirement	Points
1	Attendance	Attendance $\geq$ 75%	1
2	Internal Marks	Marks $\geq$ 45 (out of 100)	1
3	Weekly Study Hours	Planned hours $\geq$ 4	1

Final Readiness Status

The three points earned are then mapped to a final, human-readable verdict for that subject:

Conditions Met	Status Message	Interpretation
3 / 3	"Ready for exam!"	All three areas are on track.
2 / 3	"Good for exam"	One area needs attention.
1 / 3	"Not good, need more efforts"	Two areas need attention.
0 / 3	"Unacceptable, you cant pass this"	Immediate action required.

Worked Example

Consider a subject with 78% attendance, 61 internal marks and 4 planned study hours. Attendance $\geq$ 75 $\rightarrow$ point; marks $\geq$ 45 $\rightarrow$ point; hours $\geq$ 4 $\rightarrow$ point. All three conditions are satisfied, so the subject scores 3 / 3 and is marked **"Ready for exam!"**

Note: these are simple, rule-based readiness conditions defined for this project. They are not machine-learning predictions or an official academic grading policy — they exist to nudge students towards healthy attendance, performance and study habits.

4 Technologies & Python Concepts Used

Technologies

- **Python 3** — the sole language used for the entire application.
- Python's **built-in data structures** (lists and nested lists) — no external packages.
- **Text file handling** for persistent storage (no database engine required).
- A **command-line / terminal interface** — no graphical front-end.

No external Python libraries are required to run this version of the project — everything is built using the Python standard library, which keeps setup effectively down to “install Python and run the file.”

Python Concepts Demonstrated

Concept	Where it is used in this project
Variables	Store subject information and running calculations.
Lists / Nested lists	Hold multiple subjects, each as a [name, attendance, marks, hours] record.
Functions	Organise the program into add / view / check / load / save operations.
if / elif / else	Implement the readiness rules and the main-menu choices.
for loops	Process every saved subject and read each line of the data file.
while loop	Keep the main menu running until the user chooses to exit.
User input	Collect subject information interactively via <code>input()</code> .
Type conversion	Convert text input into float values for calculations.
String methods	Compare subject names (<code>.lower()</code>) and parse file lines (<code>.split()</code> , <code>.strip()</code>).
File handling	Open, read, write and close <code>subjects.txt</code> for persistence.
Exception handling	Gracefully handle a missing <code>subjects.txt</code> on first run.

5 System Design — Program Flow

The application follows a simple, linear design built around one continuous menu loop. When the program starts it attempts to load any previously saved subjects; it then repeatedly shows a four-option menu until the user chooses to save and exit.

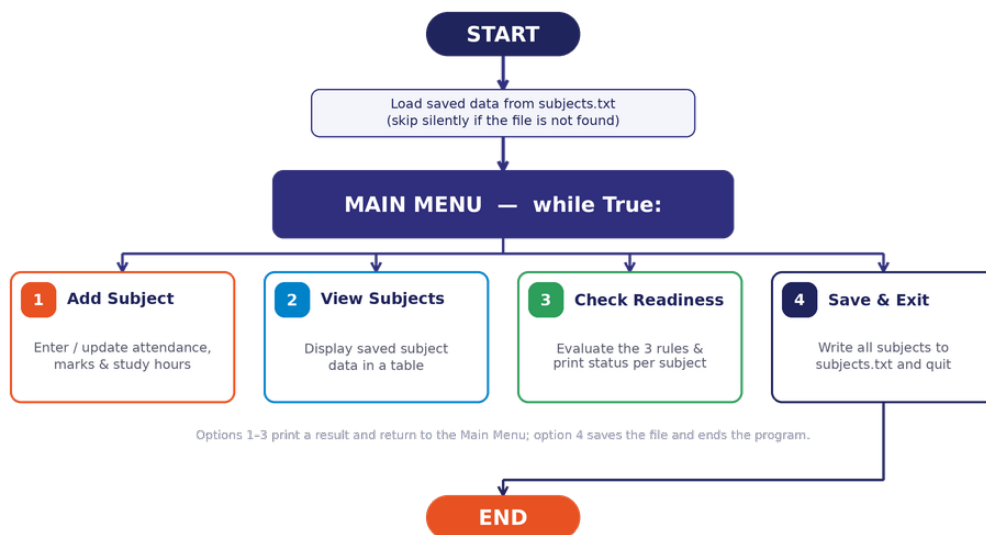


Figure 5.1 — Overall program flow of Study Buddy: Exam Readiness Checker

Each pass through the loop calls exactly one function — `add_subject()`, `view_subjects()`, `check_status()` or `save_data()` — keeping the main block short and each responsibility isolated in its own function.

The Core Loop, in Code

The whole menu system boils down to one `while True:` loop around a chain of `if / elif` checks:

```
while True:
    choice = input("Enter choice: ")
    if choice == "1":
        add_subject()
    elif choice == "2":
        view_subjects()
    elif choice == "3":
        check_status()
    elif choice == "4":
        save_data()
        break
    else:
        print("Invalid choice, try again.")
```

6 Project Structure & Data Storage Format

Repository Layout

```
study-buddy/  
|-- studybuddy.py      # complete program: menu, functions, file I/O  
|-- subjects.txt       # auto-created / updated data file  
`-- README.md          # project documentation
```

Data Storage Format

Subject records are stored one per line in `subjects.txt`, as comma-separated values in a fixed order:

```
Subject Name, Attendance, Internal Marks, Weekly Study Hours
```

Example file contents:

```
Python,85,72,5  
Mathematics,78,61,4  
English,92,80,6
```

On start-up, `load_data()` reads this file line by line, splits each line on the comma, converts the numeric fields to `float`, and appends the resulting record to the in-memory `subjects` list. If the file does not exist yet, a `FileNotFoundError` is caught and the program simply starts with an empty list — no crash, no error message shown to the user.

7 Sample Execution & Output

The screenshot below shows a real run of the program in Visual Studio Code's integrated terminal: two subjects are added (including some deliberately weak attendance and marks for one of them), and the readiness check is then run for all saved subjects.

```
PS C:\Users\irishi\Downloads> & "C:\Users\irishi\AppData\Local\Programs\Python\Python314\python.exe" "C:\Users\irishi\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher"
"main.py" - "C:\Users\irishi\Downloads\studybuddy.py"
3. Check exam readiness
4. Save and exit
Enter choice: 1
Enter subject name: 1
Enter attendance %: 89
Enter internal marks (out of 100): 76
Enter study hours planned this week: 6
Subject added.

----- Study Buddy: Exam Readiness Checker -----
1. Add subject
2. View subjects
3. Check exam readiness
4. Save and exit
Enter choice: 1
Enter subject name: eng
Enter attendance %: 23
Enter internal marks (out of 100): 10
Enter study hours planned this week: 3
Subject added.

----- Study Buddy: Exam Readiness Checker -----
1. Add subject
2. View subjects
3. Check exam readiness
4. Save and exit
Enter choice: 3

Subject Readiness:
math -> Ready for exam! ( 3 / 3 conditions met )
1 -> Ready for exam! ( 3 / 3 conditions met )
eng -> Unacceptable, you cant pass this! ( 0 / 3 conditions met )

----- Study Buddy: Exam Readiness Checker -----
1. Add subject
2. View subjects
3. Check exam readiness
4. Save and exit
Enter choice: 1
```

Figure 7.1 — Adding subjects and checking exam readiness in the terminal

Reading the Output

- A subject scoring **3 / 3** (attendance ≥ 75 , marks ≥ 45 , hours ≥ 4) is marked “*Ready for exam!*” — as seen for the subjects added with strong attendance and marks.
- A subject failing every condition — low attendance, low marks and low study hours — is flagged “*Unacceptable, you cant pass this*”, giving the student an unmistakable warning.
- Choosing **option 3** from the menu re-evaluates every saved subject in one go, making it easy to get a full picture before an exam.

8 Error Handling & Testing Notes

Built-in Safeguards

- **Missing data file:** if `subjects.txt` does not exist when the program starts, `load_data()` catches the `FileNotFoundError` and simply begins with an empty subject list instead of crashing.
- **Duplicate subjects:** before adding a new subject, `find_subject_index()` checks whether it already exists (case-insensitively) and, if so, asks the user whether to update it rather than silently creating a duplicate entry.
- **Invalid menu choice:** any input other than 1–4 at the main menu is caught by the final `else` branch, which prints a friendly “Invalid choice, try again.” instead of stopping the program.
- **Empty subject list:** both `view_subjects()` and `check_status()` check `len(subjects) == 0` first and print a clear “No subjects added yet.” message.

The Missing-File Guard, in Code

The most important safeguard is the very first thing that runs — loading data before the menu even appears. Wrapping the file read in a `try / except` block means a student's very first run (with no `subjects.txt` yet) works exactly as smoothly as the hundredth:

```
def load_data():
    try:
        file = open(FILENAME, "r")
        for line in file:
            ... # parse and append each subject
        file.close()
    except FileNotFoundError:
        pass # first run – start with an empty list, no crash
```

Manual Test Cases

Scenario	Input	Expected Result
All conditions met	Attendance 85, Marks 72, Hours 5	3/3 — “Ready for exam!”
Two conditions met	Attendance 78, Marks 61, Hours 2	2/3 — “Good for exam”
One condition met	Attendance 40, Marks 50, Hours 2	1/3 — “Not good, need more efforts”
No conditions met	Attendance 23, Marks 10, Hours 3	0/3 — “Unacceptable, you cant pass this”
Duplicate subject name	Re-adding an existing subject	Prompted to update or skip
First run, no data file	Program launched fresh	Starts with an empty subject list, no crash

9 Limitations of the Current Version

The current version is intentionally built around simple, fundamental Python concepts, which means it also carries a few deliberate limitations:

- No graphical user interface — interaction happens entirely through the terminal.
- No database — all data lives in a single flat text file.
- Input validation for out-of-range numeric values (for example, attendance above 100%) can be improved.
- The three readiness rules and their thresholds are fixed in the code, not configurable by the user.
- The program does not calculate an actual final examination grade or percentage.
- No machine-learning or predictive modelling is used — the logic is entirely rule-based.
- It does not automatically generate a detailed weekly study timetable.
- Data is stored as plain comma-separated text rather than a structured format such as CSV or JSON.

None of these limitations affect the program's core purpose; they simply mark clear directions for future improvement, several of which are outlined in the next section.

10 Future Scope

Study Buddy was deliberately kept simple for this stage of learning, but its structure leaves plenty of room to grow. Planned or possible extensions include:

- **Input validation** for attendance, marks and study-hour ranges (e.g. reject negative values).
- A more detailed **performance-prediction system** using weighted scoring.
- **Subject-wise risk levels** such as Low, Medium and High instead of a single message.
- **Automatic weekly study-hour recommendations** based on how far a subject is from readiness.
- Migrating storage to **CSV or JSON** for easier integration with other tools.
- A **graphical user interface** (Tkinter or a web front-end) for a friendlier experience.
- **Charts** (e.g. with matplotlib) to visualise subject performance over time.
- A machine-learning model, if such concepts are introduced later in the course.
- Splitting the single file into modules such as `anaLyzer.py`, `recommender.py` and `storage.py` as the project grows in scope.

11 Conclusion

Study Buddy – Exam Readiness Analyzer is a simple command-line Python project that helps students monitor their subject-wise exam readiness using attendance, internal marks and planned study hours.

The project brings together fundamental Python concepts — functions, lists, loops, conditional statements, user input, string processing and file handling — into one practical, working application, following the flow: **Input** → **Processing** → **Decision Making** → **Output** → **File Storage**.

Beyond satisfying the requirements of the **Python Essentials** course on **VITYarathi**, it reflects a problem every student can relate to, and offers a template that can be extended into a considerably more capable academic performance and study-planning tool in future iterations.

4 Core Functions (add / view / check / save)	3 Readiness Rules evaluated per subject	1 Data File subjects.txt	100% Standard-Library Python — no installs
---	--	---------------------------------------	---

Submission Detail	Value
Project Name	Study Buddy – Exam Readiness Analyzer
Submitted By	H. S. Preetham
College	VIT Bhopal University
Course / Platform	Python Essentials on VITYarathi
Repository	Public GitHub repository (root URL submitted on VITYarathi)

This report, the accompanying source code (studybuddy.py) and the project README together form the complete submission for evaluation.

COURSE COMPLETION CERTIFICATE

Awarded to H. S. Preetham · Python Essentials · VITyarthi



YOUR LEARNING DESTINATION

A RECOGNITION OF ACHIEVEMENT

Certificate of Completion

This certificate is awarded to

H.S Preetham

for successfully completing

Python Essentials

This is to certify that the awardee of this certificate has successfully completed the course mentioned above by completing all its lessons, assignments, and quizzes.

Dr. T. B. Sridharan

Pro Vice-Chancellor
VIT Bhopal University

23 Sep 2026

DATE OF COMPLETION



SCAN TO VERIFY AUTHENTICITY

<https://admin.vityarthi.com/certificate/sQLJp5rP9fLW>

VITYARTHI



Learning.
Accomplished.

CERTIFICATE OF
COMPLETION

CREDENTIAL ID

sQLJp5rP9fLW